Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development and DevOps Lab

Experiment 03 - LO4

Date of Performance: 28/07/2026 **Date of Submission: 28/07/2026**

Class: T12

Student Roll No: 2401035 **Student Name:Mayank Heda**

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Aim: To deploy multi-service applications using Docker Compose.

Theory

- **Docker Compose**: A tool for defining and running multi-container Docker applications. It uses a single YAML configuration file (docker-compose.yml) to configure application services, networks, and volumes.
- **Multi-Service Architecture:** Instead of deploying a single monolithic container, complex applications are broken down into microservices (e.g., Web Server, Database, Cache, Management Interface) running in isolated, interconnected containers.
- **Port Mapping (ports):** Binds a container's internal port to a host machine port. Custom host ports prevent conflicts with services already running locally on the host machine.
- **Data Persistence (volumes):** Containers are ephemeral by default. Docker Volumes persist data generated by containers (such as MySQL databases) even if the container is stopped or recreated.
- **Service Dependencies (depends_on):** Defines startup order dependencies between services, ensuring prerequisites (like a database) start before dependent services (like a database UI).

Application Architecture

1. **NGINX (webserver):** Web server mapped to host port 8080.
2. **MySQL 8.0 (database):** Relational database with a persistent volume (mysql-data) mapped to host port 3307 to avoid local port 3306 conflicts.
3. **Redis (rediscache):** In-memory data store for caching running on port 6379.
4. **PhpMyAdmin (db-admin):** Web-based database management interface mapped to host port 8082 to avoid host port conflicts.

Steps to create a Multi Service Architecture:-

- **Workspace Setup:** Create a dedicated project directory using mkdir compose-demo and navigate into it using cd compose-demo to keep configuration files organized.
- **Configuration File Creation:** Open a text editor and create a file named docker-compose.yml in your project folder.
- **Service Definition:** List each microservice component (e.g., webserver, database, rediscache, db-admin) along with its official Docker image name and container name.
- **Port Mapping Setup:** Map container ports to distinct host ports using the HOST_PORT:CONTAINER_PORT format (e.g., 8080:80, 3307:3306) to avoid port conflicts with applications on your host machine.
- **Environment Variable Declaration:** Pass credentials and service links under environment (e.g., setting MYSQL_ROOT_PASSWORD and pointing PMA_HOST to the database service).
- **Persistent Storage Attachment:** Define and attach a named Docker volume (e.g., mysql-data:/var/lib/mysql) under the database service so database storage survives container shutdowns.
- **Dependency Sequencing:** Include depends_on directives to ensure foundational backend services (like MySQL) launch before dependent services (like PhpMyAdmin).
- **Stack Deployment:** Execute docker compose up -d in your terminal to build the isolated network, download images, and start all containers in the background.
- **Status Verification:** Run docker compose ps to verify that all services are in an **Up**

state and check runtime logs with docker compose logs.

- **Browser Endpoint Testing:** Test service accessibility through a web browser by visiting host URLs like http://localhost:8080 (NGINX) and http://localhost:8082 (PhpMyAdmin).
- **Stack Teardown:** Execute docker compose down when finished to gracefully stop containers and remove created virtual networks.

Configuration File (docker-compose.yml)

```
version: '3.8'

services:
  # Service 1: Web Server
  webserver:
    image: nginx:latest
    container_name: webserver
    ports:
      - "8080:80"
    restart: always

  # Service 2: Database Server
  database:
    image: mysql:8.0
    container_name: mysqlserver
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: studentdb
    MYSQL_USER: student
    MYSQL_PASSWORD: student123
    ports:
      - "3307:3306"
    volumes:
      - mysql-data:/var/lib/mysql
    restart: always

  # Service 3: Cache Service
  rediscache:
    image: redis:latest
    container_name: rediscache
    ports:
      - "6379:6379"
    restart: always

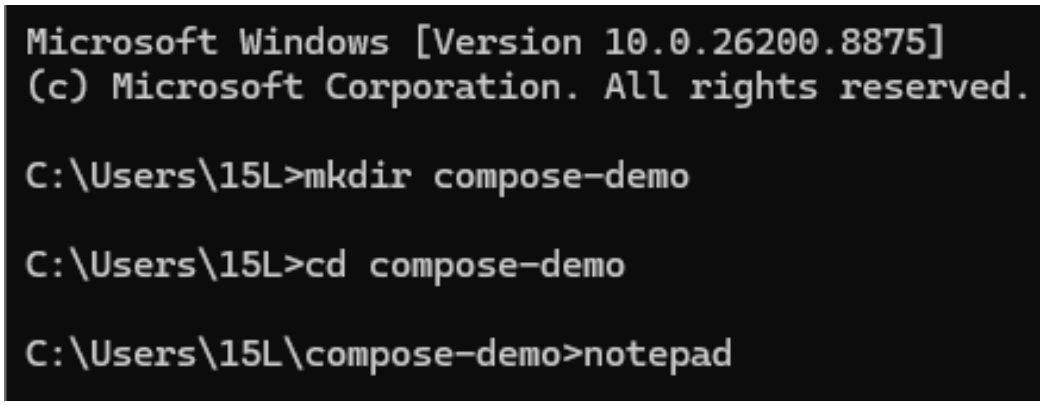
  # Service 4: Database Management Tool (GUI)
  db-admin:
    image: phpmyadmin/phpmyadmin
    container_name: phpmyadmin
    environment:
      PMA_HOST: database
      MYSQL_ROOT_PASSWORD: root123
    ports:
      - "8082:80"
    depends_on:
      - database
```

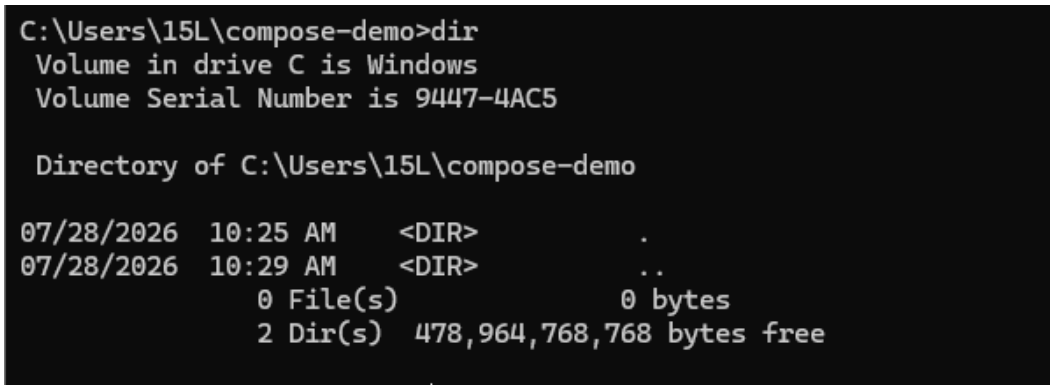
```
    restart: always

volumes:
  mysql-data:
```

Procedure & Command Execution

1. **Directory Setup & Configuration Creation:** Created project directory compose-demo and saved the multi-service docker-compose.yml file.





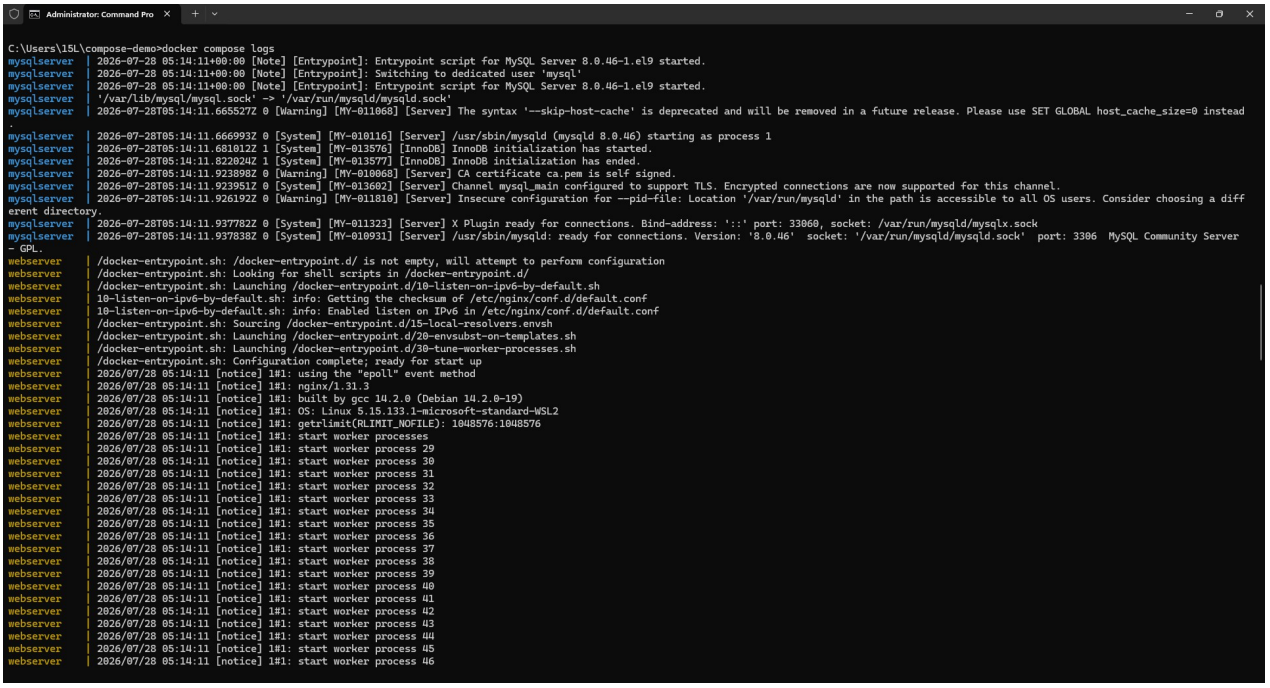
2. **Starting the Multi-Container Stack:** Launched all services in detached mode.



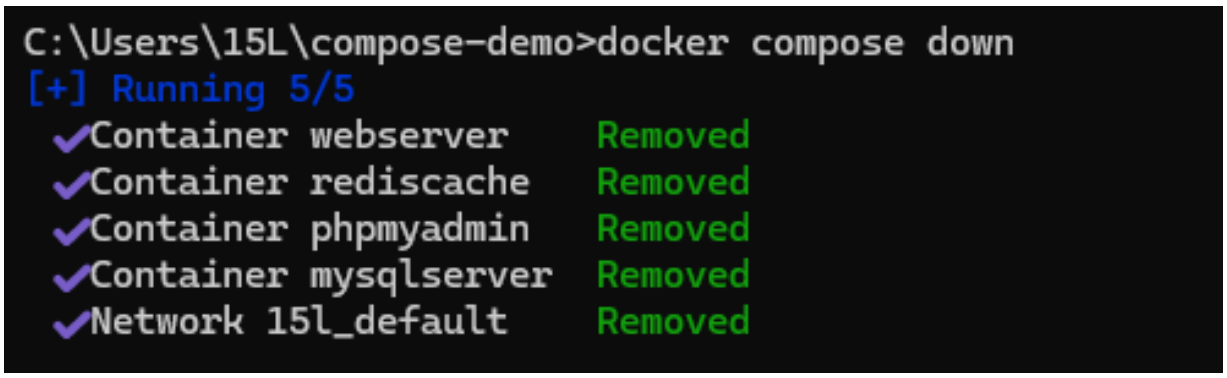
3. **Verifying Active Stack:** Checked the operational status and port bindings of all containers.

```
C:\Users\15L\compose-demo>docker compose ps
NAME          IMAGE                  COMMAND                  SERVICE      CREATED          STATUS
      PORTS
mysqlserver   mysql:8.0              "docker-entrypoint.s…"   database     55 seconds ago   Up 54 sec
onds   33060/tcp, 0.0.0.0:3307->3306/tcp
phpmyadmin    phpmyadmin/phpmyadmin  "/docker-entrypoint.…"   db-admin     55 seconds ago   Up 54 sec
onds   0.0.0.0:8082->80/tcp
rediscache    redis:latest           "docker-entrypoint.s…"   rediscache   55 seconds ago   Up 54 sec
onds   0.0.0.0:6379->6379/tcp
webserver     nginx:latest           "/docker-entrypoint.…"   webserver    55 seconds ago   Up 54 sec
onds   0.0.0.0:8080->80/tcp
```

4. **Monitoring Container Logs:** Inspected initialization and runtime logs across the stack.



5. **Environment Teardown:** Cleanly stopped and removed containers and networks after verification.

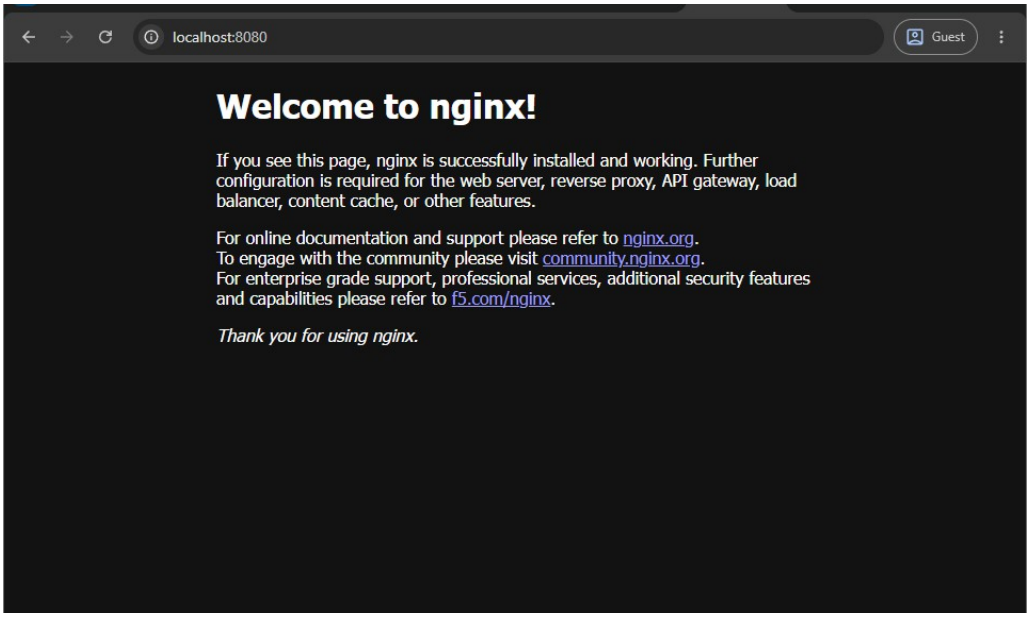


Browser Verification Results

1. Web Server Verification (NGINX)

- **URL:** http://localhost:8080

- **Observation:** The default NGINX welcome page successfully loaded in the browser.
- **Explanation:** This confirms that the webserver container is running properly, listening on internal port 80, and correctly mapped to host port 8080.



2. Database GUI Verification (PhpMyAdmin)

- **URL:** http://localhost:8082

- **Observation:** The PhpMyAdmin login page loaded successfully.
- **Credentials Used:**
    - **Username:** student

    - **Password:** student123

- **Explanation:** This verifies that the phpmyadmin container is running on port 8082 and successfully communicating over the internal Docker network with the mysqlserver container running on port 3307.

3. Verify Redis (In-Memory Cache)

```
C:\Users\15L\compose-demo>docker exec -it rediscache redis-cli ping
PONG
```

Conclusion

- **Container Orchestration:** Gained hands-on experience using **Docker Compose** to manage multiple interdependent microservices as a single stack using a single docker-compose.yml configuration file.
- **Port Conflict Resolution:** Learnt how to resolve host network port conflicts by remapping host ports (e.g., mapping host 3307 to container 3306, and host 8082 to container 80) without altering the internal container configurations.
- **Data Persistence:** Understood how Docker Volumes (mysql-data) allow database state to persist independently of the container lifecycle, preventing data loss when containers are recreated or stopped.
- **Container Networking & Inter-service Communication:** Experienced how Docker automatically establishes an internal network (compose-demo_default), allowing containers like phpmyadmin to talk directly to database using their service names.
- **Lifecycle Management:** Mastered essential Docker Compose CLI commands for starting (up -d), inspecting (ps), logging (logs), and tearing down (down) multi-container environments.